TEACHING GUIDE

Getting Started with Lean

What a proof assistant is, a first proof checked by a computer, and where to go next.

Alessandro Linzi · November 2026

What Lean is

A **proof assistant** is a program that checks proofs. You write each step in a precise language, and the computer accepts the proof only when every step follows from the previous ones. Nothing is left as "clearly" or "similarly".

Lean 4 is an open-source programming language and proof assistant. **Mathlib** is its community library of formalized mathematics, from algebra and number theory to analysis and topology. When you formalize a proof, you find out exactly what it needs: which hypothesis is used where, which case you forgot, which step you only believed.

A first proof: $0 + n = n$

In Lean the natural numbers are defined by the Peano axioms: every natural number is either 0 or the successor $k + 1$ of another one, and addition is defined by recursion on the second argument, so $m + (k + 1) = (m + k) + 1$. The statement $0 + n = n$ is therefore not immediate; it needs induction on n.

```
theorem my_zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih]
```

Read it line by line:

- `induction n with` splits the proof into the two cases of the induction principle.
- `| zero => rfl`: the base case $0 + 0 = 0$ holds by the definition of addition, so `rfl` ("both sides are the same by definition") closes it.
- `| succ k ih => ...`: we must prove $0 + (k + 1) = k + 1$, and `ih` is the induction hypothesis $0 + k = k$. The rewrite `Nat.add_succ` turns the left side into $(0 + k) + 1$, and rewriting with `ih` turns it into $k + 1$. The goal is closed.

When a step is wrong, Lean says so. Replace the last line by `| succ k ih => rfl` and Lean rejects the proof:

```
error: Tactic `rfl` failed: The left-hand side
  0 + (k + 1)
is not definitionally equal to the right-hand side
  k + 1
```

The error points at the exact step that does not follow. That is the whole idea: you can never fool the checker, so a proof it accepts has no gaps.

Both versions were checked with Lean 4.35.0 (core library only, no Mathlib) in November 2026. Lean already contains this fact as `Nat.zero_add`; we prove our own copy to see how it works. The Natural Number Game

builds the same result step by step, in its own simplified style.

A learning path

1. **Play the Natural Number Game** in the browser. It rebuilds the natural numbers from the Peano axioms and teaches the basic tactics. It is available in Italian too.
2. **Try more games** on the Lean Game Server, for example Scribble, which covers first-year university topics, or the Set Theory Game.
3. **Experiment in Lean 4 Web**, the official web editor, which has Mathlib: paste the proof above and change it.
4. **Install Lean** on your computer (VS Code with the Lean 4 extension) when you want to work on longer files.
5. **Work through Mathematics in Lean**, the standard tutorial for doing mathematics with Mathlib.
6. **Go deeper** with *Theorem Proving in Lean 4* (how Lean works underneath) or Kevin Buzzard's *Formalising Mathematics* handbook (written for final-year undergraduates).
7. **When you are stuck,** search Mathlib with Loogle, and ask in the new members channel of the Lean Zulip chat.

Links

Natural Number Game	adam.math.hhu.de/#/g/leanprover-community/nng4
Lean Game Server	adam.math.hhu.de
Lean 4 Web	live.lean-lang.org
Install Lean	lean-lang.org/install
Mathematics in Lean	leanprover-community.github.io/mathematics_in_lean
Theorem Proving in Lean 4	lean-lang.org/theorem_proving_in_lean4
Formalising Mathematics (K. Buzzard)	ma.imperial.ac.uk/~buzzard/xena/formalising-mathematics-2024
Mathlib and its documentation	mathlib.org, leanprover-community.github.io/mathlib4_docs
Loogle (Mathlib search)	loogle.lean-lang.org
Lean Zulip chat	leanprover.zulipchat.com

Last checked: November 2026.